



武汉理工大学
WUHAN UNIVERSITY OF TECHNOLOGY

课程：企业级应用软件设计与开发

项目名称：AI 游戏教练

方向：Agentic AI 原生开发

姓名：张妍妍

学号：2025302993

专业：计算机技术

指导老师：戚欣

2026 年 6 月 22 号

目录

摘 要	III
1 选题背景与设计思想	1
1.1 问题定义	1
1.2 现有方案不足	1
1.3 项目价值	2
1.3.1 业务价值	2
1.3.2 学习价值	2
2 Specs 规格文档	3
2.1 Product Spec	3
2.1.1 产品定位	3
2.1.2 目标用户	3
2.1.3 核心功能需求	4
2.1.4 非功能需求	5
2.2 Architecture Spec	6
2.2.1 整体架构模式	6
2.2.2 模块划分	6
2.2.3 核心技术约束	7
2.3 API Spec	8
2.3.1 用户交互接口	8
2.3.2 记忆读写接口	8
2.3.3 工具调用接口	8
2.3.4 日志监控接口	9
3 系统架构与设计	10
3.1 整体系统架构	10
3.1.1 用户交互层	11
3.1.2 智能体核心层	11
3.1.3 能力支撑层	11
3.1.4 数据存储层	12
3.2 Agent 交互流程	12

3.3 核心数据流设计	13
4 关键实现与代码展示。	16
4.1 Agent 核心循环	16
4.1.1 状态结构体定义	17
4.1.2 循环节点与流转逻辑	17
4.1.3 完整执行流程说明	17
4.2 工具定义	18
4.2.1 入参 Schema 与工具代码	19
4.2.2 工具调用绑定	19
4.3 配置文件	19
4.3.1 .env 环境配置文件	20
4.3.2 config.py 全局读取配置	20
4.3.3 llm/client.py Provider 分发	20
4.3.4 llm/deepseek_client.py 调用封装	21
4.4 AI IDE 使用截图说明	21
5 测试与评估	24
5.1 测试环境	24
5.2 测试方法与覆盖范围	24
5.3 功能测试结果	24
5.4 接口与模型调用测试	29
5.5 Agent 行为评估	29
5.6 Benchmark 基准测试	30
5.7 Demo 截图与可视化证据	30
6 系统升级与扩展	34
6.1 现有架构可扩展性分析	34
6.2 下一阶段迭代计划	35
6.3 AI 能力演进路径	36
7 课程总结	38
7.1 个人收获	38
7.2 工程思维转变	38
7.3 对课程的建议	39

摘 要

随着大语言模型和 Agent 技术的快速发展，基于智能体的 AI 原生应用逐渐成为人工智能领域的重要研究方向。传统游戏攻略查询方式存在信息分散、交互性不足以及缺乏个性化指导等问题，难以满足玩家在真实对局场景中的决策需求。针对上述问题，本文设计并实现了一套面向《王者荣耀》场景的 AI 游戏教练系统，通过融合大语言模型、知识库检索、用户记忆管理以及 ReAct 推理机制，为玩家提供智能化游戏辅助服务。

本系统采用 FastAPI 构建后端服务，以 DeepSeek 大模型作为核心推理引擎，通过英雄知识库、攻略知识库和装备知识库提供领域知识支撑。同时，引入用户长期记忆机制记录玩家偏好信息，实现个性化推荐功能。在系统设计过程中，采用 ReAct 推理模式，将系统决策过程划分为 Thought、Action 和 Observation 三个阶段，并以可视化形式向用户展示推理链，提高回答结果的可解释性和可信度。此外，系统还集成英雄推荐、攻略查询、出装建议、阵容分析以及历史对话管理等功能，构建完整的电竞教练服务闭环。

在实现过程中，本文完成了系统需求分析、架构设计、Agent 工作流设计、知识库构建以及接口开发，并通过功能测试、接口测试和自动化测试对系统进行验证。测试结果表明，系统能够稳定完成知识检索、工具调用、模型推理以及结构化结果生成等任务，满足预期设计目标。

本项目不仅验证了 Agentic AI 在游戏辅助场景中的应用可行性，也探索了 AI IDE 协同开发模式在智能体应用开发过程中的实践价值，为未来 AI 原生应用设计与实现提供了参考。

关键词： Agentic AI； AI 游戏教练； DeepSeek； ReAct； 知识库检索； FastAPI

1 选题背景与设计思想

1.1 问题定义

《王者荣耀》作为典型多人在线竞技手游，具有英雄数量多、分路体系复杂、版本节奏变化快、团战博弈强和对局决策实时性高等特点。玩家在排位上分过程中普遍面临多项痛点：一是新手难以快速理解打野、中路、发育路、对抗路和辅助等位置职责；二是英雄选择、出装铭文、开局路线、Gank 时机、暴君/主宰资源交换等决策成本高；三是攻略信息分散在社区和视频平台，且难以结合当前英雄、段位、局势和敌方阵容给出即时判断。

针对以上痛点，本项目最终聚焦《王者荣耀》单一游戏场景，开发一款“AI 游戏教练系统”。系统不再停留于普通问答或静态攻略，而是围绕历史对话、对局信息采集、工具化面板和 ReAct 推理可视化构建交互闭环，帮助玩家在英雄练习、出装铭文、团战决策、逆风处理和对局复盘中获得更像真人教练的连续策略指导。

1.2 现有方案不足

目前市场内游戏辅助相关产品主要分为通用图文攻略平台、固定脚本辅助工具、通用问答 AI 三类，各类方案均存在明显短板：

（1）图文攻略平台：以 B 站、贴吧、小黑盒、游侠网为代表，内容由用户或创作者静态发布，内容更新滞后、检索效率低，仅能提供标准化攻略，无法结合用户实时游戏进度、游玩习惯动态调整决策方案；

（2）传统脚本辅助工具：多为固定指令自动化脚本，仅支持简单的挂机、资源采集操作，无自主推理能力，无法应对动态游戏场景，且适配品类单一，扩展性极低；

（3）通用大模型问答：普通 LLM 仅支持单次简单问答，无长期对话记忆、多步骤推理能力，无法连续跟踪用户游玩全流程，也不能调用工具完成数据查询、方案对比等复杂操作，难以满足深度游玩决策需求。

1.3 项目价值

1.3.1 业务价值

本项目打造《王者荣耀》专属游戏 AI 教练，打破传统攻略产品静态化、通用化的局限性。系统既能回答英雄与分路问题，也能通过历史对话框保留上下文，通过信息采集按钮快速收集当前英雄、位置、段位、局势、敌方核心输出和玩家意图，并在回答中展示 Thought、Action、Observation 三段式推理过程。相比全品类泛化助手，单游戏聚焦能够显著提升知识库质量、推荐准确率和临场决策体验。

1.3.2 学习价值

项目严格遵循 SDD 规格驱动开发全流程，整合四大核心技术：SDD 规格驱动开发、Function Calling 工具调用、向量数据库记忆机制、基于 LangGraph 的状态管理与多步骤推理，完整落地 Agentic AI 原生智能体开发流程，深度理解智能体编排逻辑、工程化开发模式，完成从代码编写者向智能体编排者的思维转型。

本项目采用 Python 作为开发语言，基于 FastAPI 提供 Web UI 与 API 服务，默认接入 DeepSeek API 作为主要推理模型，并保留 Ollama 本地 qwen2.5:3b 作为可选备用。系统配合 Chroma 长期记忆、JSON 玩家偏好、王者荣耀英雄知识库、工具化按钮面板和 ReAct 推理可视化，实现从“问答式攻略”到“电竞 AI 教练”的能力升级。

DeepSeek 接入采用 OpenAI Compatible API 方式完成：后端通过 `LLM_PROVIDER=deepseek` 选择 `DeepSeekProvider`，使用 `DEEPSEEK_BASE_URL=https://api.deepseek.com`、`DEEPSEEK_MODEL=deepseek-chat` 和 `DEEPSEEK_API_KEY` 完成鉴权与模型调用；`llm/client.py` 负责 Provider 路由，`llm/deepseek_client.py` 使用 OpenAI SDK 创建客户端并调用 `chat.completions.create`，从而保证 DeepSeek 与 Ollama 两套模型后端可以在不修改业务 Agent 逻辑的前提下自由切换。

2 Specs 规格文档

2.1 Product Spec

2.1.1 产品定位

本项目开发的“AI 游戏教练系统——王者荣耀”是一款基于大语言模型的 Agentic AI 原生应用。系统以《王者荣耀》游戏场景为核心，通过结合知识库检索、用户记忆管理、工具调用以及 ReAct 推理机制，为玩家提供英雄推荐、阵容分析、出装建议、攻略查询以及对局决策辅助等服务。

与传统游戏攻略网站相比，本系统不仅能够回答用户问题，还能够结合玩家偏好、历史对话以及当前游戏场景进行分析推理，并生成结构化、可解释的决策建议，从而实现“电竞 AI 教练”的功能定位。产品定位说明的表格如 2-1 所示：

表 2-1：产品定位说明

项目	内容
产品名称	电竞 AI 教练系统——王者荣耀智能助手
产品类型	Agentic AI 原生应用
目标领域	MOBA 电竞游戏辅助
核心能力	智能问答、英雄推荐、阵容分析
推理模型	DeepSeek
部署方式	FastAPI + Web UI
使用方式	浏览器访问

2.1.2 目标用户

本系统主要面向以下用户群体：

（1）《王者荣耀》普通玩家

针对刚接触游戏的新手玩家，系统能够提供英雄推荐、技能介绍、出装建议以及基础对局指导等服务，帮助用户快速熟悉游戏机制。

（2）进阶玩家

针对具有一定游戏经验的玩家，系统能够结合当前局势进行阵容分析、英雄克制分析以及团战决策辅助，提高玩家对游戏理解和决策能力。

（3）电竞爱好者

针对关注赛事和高端对局的玩家，系统能够提供战术分析、版本强势英雄推荐以及运营思路参考等内容，帮助用户提升竞技水平。

目标用户的分析如表 2-2 所示：

表 2-2：目标用户分析

用户类型	特征	核心需求
新手玩家	刚接触游戏	学习英雄与基础操作
普通玩家	日常排位	提升胜率
高阶玩家	巅峰赛玩家	阵容分析
电竞爱好者	关注赛事	战术学习

2.1.3 核心功能需求

根据系统定位，本项目需要实现以下核心功能：

（1）智能问答功能：支持自然语言交互，能够理解用户提出的游戏相关问题，并生成专业化回答。

（2）英雄推荐功能：根据玩家需求和游戏场景，从英雄知识库中筛选合适英雄，并给出推荐理由。

（3）攻略查询功能：支持游戏攻略检索，为玩家提供发育路线、团战技巧以及运营思路等内容。

（4）出装推荐功能：结合英雄定位和当前版本信息，提供合理的装备推荐方案。

（5）用户记忆功能：记录用户偏好英雄、常用位置以及游戏风格，实现个性化服务。

（6）ReAct 推理展示功能：展示系统的分析过程和决策依据，提高回答结果的可解释性。

系统的核心功能需求如表 2-3 所示：

表 2-3：功能需求分析

编号	功能名称	功能描述
FR-01	智能问答	回答游戏问题
FR-02	英雄推荐	推荐适合英雄
FR-03	出装推荐	推荐装备方案
FR-04	阵容分析	分析敌我阵容
FR-05	攻略查询	检索攻略内容
FR-06	历史对话	保存历史会话
FR-07	用户记忆	记录玩家偏好
FR-08	ReAct 展示	展示推理过程

2.1.4 非功能需求

为了保证系统能够稳定运行，还需要满足以下非功能需求：

- （1）可扩展性：系统应支持后续增加新的知识库、工具模块以及模型服务。
 - （2）可维护性：采用模块化设计，降低系统耦合度，方便后续维护和升级。
 - （3）可解释性：回答结果需要提供推理过程展示，而不仅仅给出最终结论。
 - （4）可用性：系统界面应简单易用，支持连续对话和历史记录管理。
 - （5）可靠性：系统应能够稳定完成模型调用、知识检索以及工具执行等任务。
- 系统的非功能需求如表 2-4 所示：

表 2-4：非功能需求表

类别	需求
性能	响应时间<5 秒
可扩展性	支持新增模型
可维护性	模块解耦
可解释性	支持 ReAct
易用性	Web 界面操作简单
安全性	API Key 环境变量管理

2.2 Architecture Spec

2.2.1 整体架构模式

本项目采用分层架构设计模式，整体由交互层、智能体核心层、能力支撑层以及数据存储层组成。系统的分层架构说明如下表 2-5 所示，该架构具有职责清晰、耦合度低以及扩展能力强等特点。

用户通过 Web UI 发起请求后，系统首先读取历史会话和用户偏好信息，然后进入 Agent 推理流程。Agent 根据问题类型调用对应知识库和工具模块，并通过 DeepSeek 大模型完成分析与决策。最终系统生成结构化结果并返回前端进行展示。

表 2-5 系统分层架构说明

层级	主要模块	职责
交互层	Web UI	用户交互
Agent 层	game_agent.py	决策中心
能力层	DeepSeek、Tools	推理与工具
数据层	Memory、Knowledge	数据存储

2.2.2 模块划分

系统主要划分为以下模块，模块职责表如下面 2-6 所示：

- （1）API 服务模块（api.py）：负责接收前端请求并返回结构化结果。
- （2）Agent 核心模块（game_agent.py）：负责意图识别、知识检索、工具调度以及结果生成。
- （3）模型调用模块（llm/client.py、deepseek_client.py）：负责管理模型路由和 DeepSeek API 调用。
- （4）记忆管理模块（memory.py、local_memory.py）：负责历史会话和用户偏好管理。
- （5）知识库模块（knowledge）：负责存储英雄、攻略以及装备相关知识。
- （6）工具模块（tools.py、hero_recommend_service.py）：负责完成英雄推荐、攻略检索以及装备推荐等任务。
- （7）日志模块（tracer.py）：负责记录系统运行日志和 Trace 信息。

表 2-6 模块职责表

模块	文件	职责
API 服务	api.py	提供接口
Agent 核心	game_agent.py	推理流程
模型服务	llm/client.py	模型路由
DeepSeek	deepseek_client.py	API 调用
记忆管理	memory.py	用户偏好
英雄推荐	hero_recommend_service.py	英雄筛选
工具模块	tools.py	工具调度
日志模块	tracer.py	Trace 记录

2.2.3 核心技术约束

系统开发过程中主要遵循以下技术约束，技术栈说明如表 2-7 所示：

- （1）后端采用 FastAPI 框架实现；
- （2）大模型采用 DeepSeek API 服务；
- （3）知识库采用 JSON 文件管理；
- （4）用户记忆采用本地文件和浏览器存储实现；
- （5）系统采用 ReAct 推理模式增强可解释性；
- （6）所有接口均采用 RESTful 风格设计。

表 2-7 技术栈说明

技术	用途
Python	开发语言
FastAPI	后端框架
DeepSeek API	大模型推理
JSON	知识存储
Swagger	接口测试
Pytest	自动化测试
Ruff	代码规范检查

2.3 API Spec

2.3.1 用户交互接口

用户交互接口是系统核心接口，负责接收用户问题并返回结构化结果。

接口地址：

POST /api/chat

输入参数：

```
{  
  "message": "推荐一个适合新手的打野英雄"  
}
```

返回结果：

```
{  
  "summary": "...",  
  "key_points": [...],  
  "details": "...",  
  "recommendations": [...],  
  "react_trace": {...}  
}
```

2.3.2 记忆读写接口

系统通过记忆管理模块实现用户偏好信息读取和保存。

主要功能包括：读取用户偏好信息、保存用户长期记忆、管理历史对话记录 and 构建用户上下文信息。记忆数据主要存储于 `player_memory.json` 文件以及浏览器 `localStorage` 中。

2.3.3 工具调用接口

Agent 在执行过程中可根据任务需要调用不同工具模块。当前支持：Hero Recommendation Tool、Guide Retrieval Tool、Equipment Recommendation Tool 和 Team Analysis Tool，其执行结果会作为上下文信息参与后续模型推理。

2.3.4 日志监控接口

系统通过 Trace 机制记录运行过程中的关键数据。日志内容包括：用户请求内容、gent 执行过程、具调用结果、型响应信息、口执行耗时和 race ID。日志信息统一保存至 logs 目录，用于系统调试和运行监控。

3 系统架构与设计

3.1 整体系统架构

本项目采用分层模块化架构设计，整体划分为用户交互层、智能体核心层、能力支撑层以及数据存储层四个部分。各层之间通过标准接口进行通信，既保证了系统功能的完整性，又提高了系统的可维护性和可扩展性。

系统以 FastAPI 作为后端服务框架，以 DeepSeek 大模型作为核心推理引擎，通过知识库检索、用户记忆管理以及 ReAct 推理机制实现电竞 AI 教练功能。用户通过 Web UI 提交问题后，系统首先进行意图识别和上下文构建，然后结合知识库内容和用户偏好完成推理分析，最终生成结构化结果返回前端展示。

整体架构遵循“用户交互—智能决策—能力支撑—数据管理”的设计思路，各层职责明确，相互协作，共同完成电竞 AI 教练系统的业务流程。整体系统架构图如 3-1 所示。

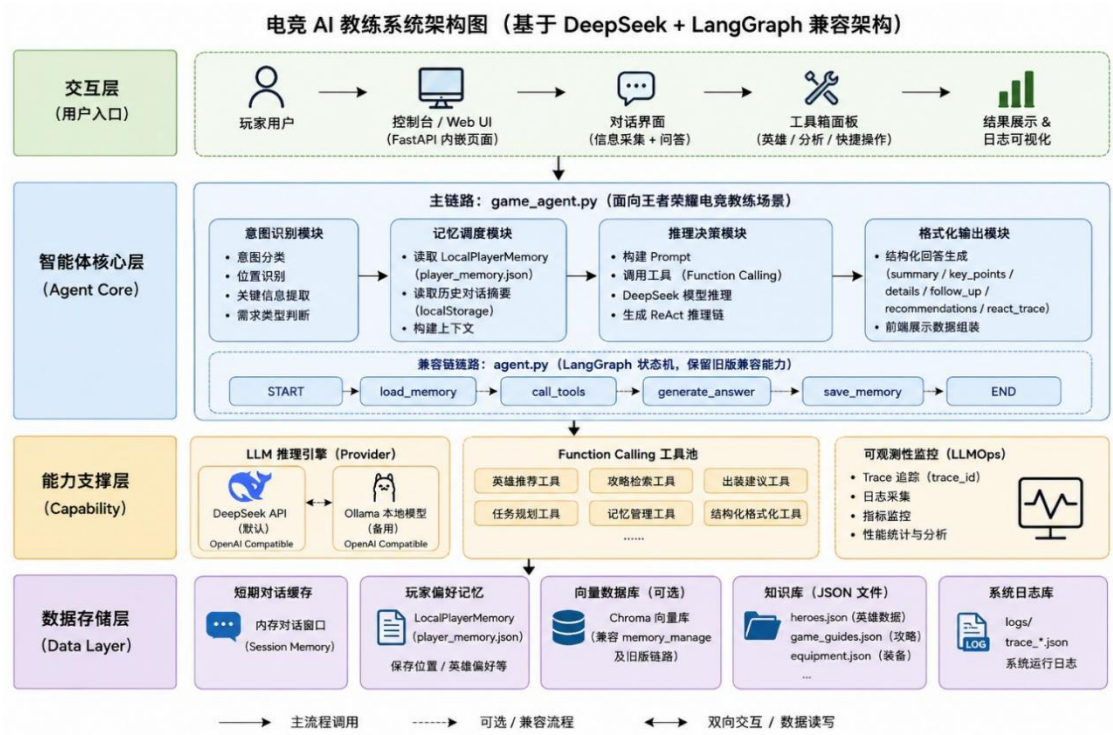


图 3-1 整体系统架构图

3.1.1 用户交互层

交互层是系统与用户直接接触的部分，主要负责用户请求接收、结果展示以及历史会话管理等功能。

用户通过 Web UI 输入游戏相关问题，例如英雄推荐、阵容分析、出装建议以及对局决策等内容。系统接收用户请求后，将问题发送至后端 Agent 进行处理，并最终将分析结果返回给用户。

为了提升用户体验，系统支持历史对话记录管理功能。用户可以查看之前的咨询内容和回答结果，实现连续性的电竞教练服务。同时，系统能够展示 ReAct 推理过程和结构化结果，使用户不仅能够获得最终答案，还能够了解系统的分析依据和决策逻辑。

交互层主要由前端页面、对话界面、历史会话模块以及结果展示模块组成。

3.1.2 智能体核心层

智能体核心层是整个系统的决策中心，负责完成意图识别、上下文构建、推理分析以及结果生成等核心任务。

当用户提交问题后，系统首先执行意图识别模块，对问题进行分类判断。例如识别当前问题属于英雄推荐、攻略查询、阵容分析还是出装建议等类型。随后系统读取用户历史偏好和会话上下文信息，构建完整的推理环境。

在推理过程中，系统采用 ReAct (Reasoning + Acting) 思想，将决策过程划分为 Thought、Action 和 Observation 三个阶段。其中 Thought 用于分析当前问题和局势，Action 用于制定具体策略和建议，Observation 用于预测执行结果和潜在影响。

智能体核心层最终将推理结果整理为统一的结构化响应格式，包括总结信息、关键要点、详细分析、推荐建议以及追问问题等内容，并返回前端进行展示。

3.1.3 能力支撑层

能力支撑层为智能体提供知识检索、模型推理以及工具调用等能力支持，是系统完成智能决策的重要基础。

在模型推理方面，系统采用 DeepSeek 大模型作为主要推理引擎，通过 OpenAI Compatible API 接口完成模型调用。系统根据用户问题和知识库内容构建 Prompt，并将其发送至 DeepSeek 模型进行推理分析。

在知识支撑方面，系统维护了多个领域知识库，包括英雄知识库（`heroes.json`）、攻略知识库（`game_guides.json`）以及装备知识库（`equipment.json`）。这些知识库为系统提供可靠的事实依据，避免完全依赖大模型生成结果。

此外，系统还集成了多个工具模块，例如英雄推荐工具、攻略检索工具、装备推荐工具以及阵容分析工具等。智能体能够根据不同任务自动调用相应工具，提高回答的准确性和专业性。

3.1.4 数据存储层

数据存储层负责管理系统运行过程中产生的各种数据，包括用户记忆、历史会话、知识库数据以及日志信息等。

用户长期偏好信息通过 `player_memory.json` 文件进行保存，主要记录用户常用位置、偏好英雄以及游戏风格等内容。历史会话记录则通过浏览器 `localStorage` 进行管理，实现会话状态的持久化保存。

系统知识库采用 JSON 文件形式进行存储，包括英雄信息、攻略内容以及装备推荐数据等。该方式具有结构简单、维护方便以及扩展灵活等特点，能够满足课程项目的实际需求。

为了支持系统调试与性能分析，系统还设计了日志记录机制，对请求内容、响应结果、推理链以及执行耗时等信息进行记录，并生成唯一的 `trace_id` 用于问题追踪和运行监控。

通过上述数据管理机制，系统能够实现用户记忆管理、知识检索以及运行状态监控等功能，为电竞 AI 教练系统的稳定运行提供数据支撑。

3.2 Agent 交互流程

为了实现对局分析和游戏决策辅助功能，本项目设计了一套基于 ReAct 推理思想的 Agent 执行流程。系统不仅需要回答用户问题，还需要能够展示分析过程，提高结果的可解释性和可信度。

当用户发起请求后，Agent 首先加载历史会话信息和用户长期偏好记忆。例如用户常用位置、偏好英雄以及游戏风格等信息都会参与后续推理过程。随后系统对用户问题进行意图识别，判断当前问题属于英雄推荐、阵容分析、攻略查询还是出装建议等类型。

在确定问题类型后，Agent 会从对应知识库中检索相关数据，并结合用户输入构建完整 Prompt。构建完成后调用 DeepSeek 模型进行推理分析，同时生成 ReAct 推理过程。其中 Thought 阶段负责分析当前局势和需求，Action 阶段负责制定具体策略或推荐方案，Observation 阶段负责预测执行结果和可能影响。

推理结束后，系统将结果转换为统一的结构化输出格式，包括 summary、key_points、details、recommendations 以及 react_trace 等字段，并记录 trace 日志信息，最终返回给前端页面进行展示。

通过上述流程，系统实现了从用户提问到智能决策生成的完整闭环，同时能够保留推理过程和历史上下文信息，为用户提供连续性的电竞教练服务。图 3-2 展示了 AI 游戏教练 Agent 的执行流程。

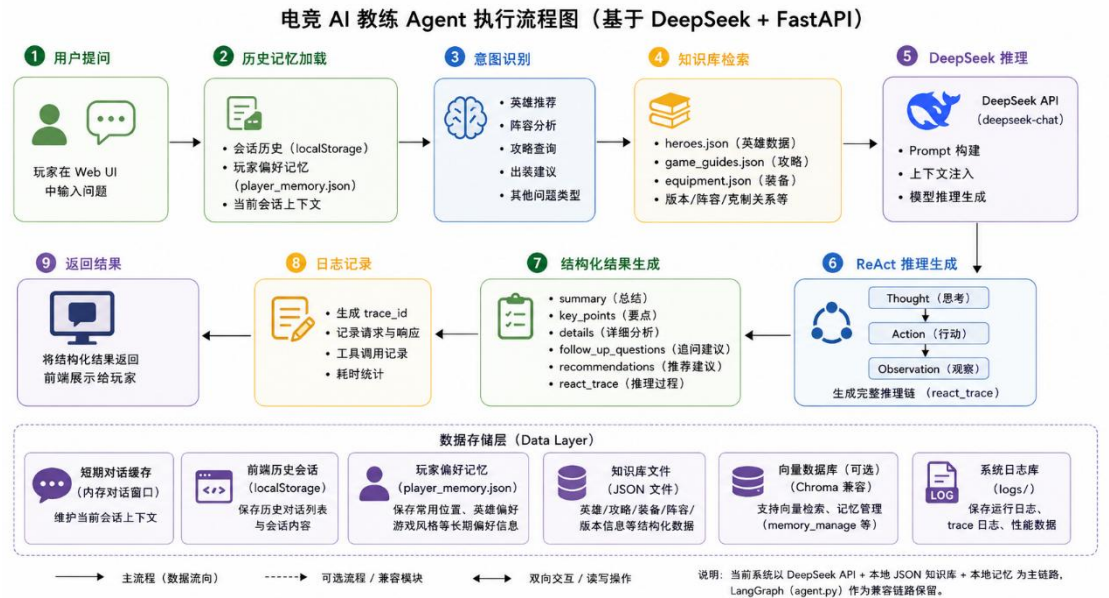


图 3-2 Agent 执行流程图

3.3 核心数据流设计

为了保证系统能够稳定完成知识检索、用户记忆管理以及大模型推理任务，本项目设计了完整的数据流转机制。核心数据流覆盖用户输入、知识检索、模型推理、结果生成以及日志记录等多个环节。

系统运行过程中，用户首先通过前端界面输入问题，数据经 FastAPI 接口进入后端服务。系统随后读取浏览器本地存储中的历史会话信息以及 player_memory.json 中保存的用户偏好数据，形成完整的上下文环境。

在完成意图识别后，系统从多个知识源中检索相关信息，包括 heroes.json 英雄知识库、game_guides.json 攻略知识库以及 equipment.json 装备知识库。检索结果与用户上下文共同构成 Prompt 输入，并提交至 DeepSeek 模型进行推理。

模型返回结果后，系统生成 ReAct 推理链数据，并进一步构建结构化响应内容，包括总结信息、关键要点、详细分析、推荐建议以及追问问题等字段。同时系统将请求信息、响应内容、推理链以及执行耗时等数据记录到日志文件中，以支持后续调试与性能分析。

最终，结构化结果返回至前端页面进行展示，用户不仅能够查看最终答案，还能够查看系统的推理过程和决策依据，从而提升交互体验和系统可信度。图 3-3 展示了系统核心数据流设计。

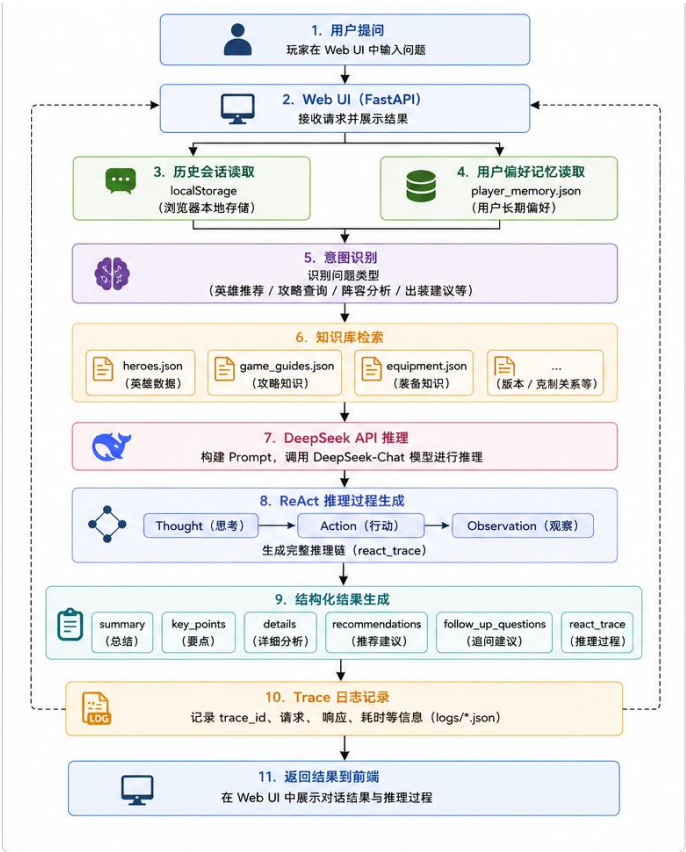


图 3-3 核心数据流设计图

由于本项目主要采用 JSON 文件和本地存储方式管理数据，而非传统关系型数据库，因此系统采用数据对象关系模型描述核心数据结构之间的关联关系。

系统中的主要数据对象包括用户偏好记忆（PlayerMemory）、历史会话记录（ConversationHistory）、英雄知识库（HeroKnowledge）、攻略知识库（GuideKnowledge）、装备知识库（EquipmentKnowledge）以及系统日志（TraceLog）等。

其中，PlayerMemory 负责保存用户长期偏好信息，例如常用位置、偏好英雄和游戏风格等；ConversationHistory 用于记录历史对话内容和会话摘要；HeroKnowledge、GuideKnowledge 和 EquipmentKnowledge 共同构成系统知识库，为后续推理提供事实依据；TraceLog 则负责保存系统运行过程中的请求、响应和推理链信息，用于调试和可观测性分析。

各数据对象之间通过业务逻辑建立关联关系，共同支撑电竞 AI 教练系统的知识检索、推理分析和连续对话功能。图 3-4 展示了系统核心数据对象关系。

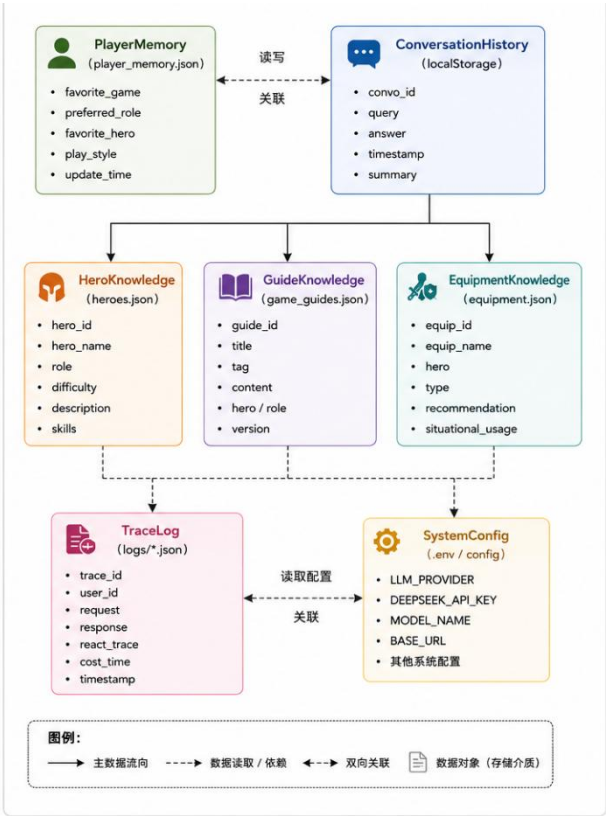


图 3-4 核心数据对象设计

4 关键实现与代码展示。

4.1 Agent 核心循环

Agent 是本系统实现智能决策与交互服务的核心组件。与传统问答系统不同，本项目中的 Agent 不仅负责生成回答，还需要完成用户意图识别、知识检索、工具调用、推理分析以及结果组织等多个环节。为了提高回答质量和可解释性，系统采用 ReAct 思想构建 Agent 执行流程。

在运行过程中，Agent 首先接收用户输入，然后结合历史会话和用户偏好信息进行上下文构建。在此基础上，系统完成问题分类和知识检索，并调用 DeepSeek 模型进行推理分析。推理结束后生成结构化回答以及 ReAct 推理链，最终返回给前端进行展示。图 4-1 展示了 Agent 核心执行流程。

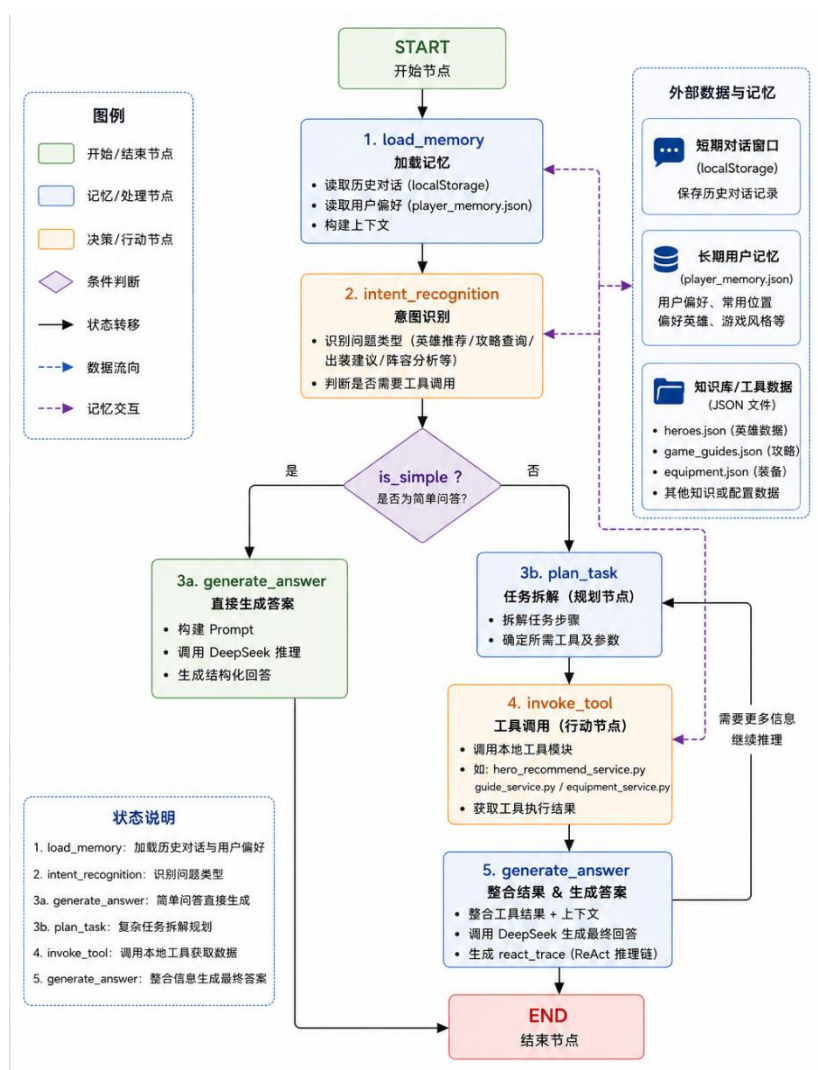


图 4-1 Agent 核心执行流程

4.1.1 状态结构体定义

```
✓ class AgentState(TypedDict):  
    user_query: str  
    user_id: str  
    game_name: str  
    task_type: str  
    game_stage: str  
    play_style: str  
    favorite_character: str  
    current_goal: str  
    trace_id: str  
    short_memory: list[dict[str, str]]  
    long_memory: list[str]  
    tool_results: list[dict[str, Any]]  
    final_answer: str  
    memory_saved: bool  
    trace: AgentTrace
```

4.1.2 循环节点与流转逻辑

```
def build_graph():  
    graph = StateGraph(AgentState)  
    graph.add_node("load_memory", load_memory_node)  
    graph.add_node("call_tools", call_tools_node)  
    graph.add_node("generate_answer", generate_answer_node)  
    graph.add_node("save_memory", save_memory_node)  
  
    graph.add_edge(START, "load_memory")  
    graph.add_edge("load_memory", "call_tools")  
    graph.add_edge("call_tools", "generate_answer")  
    graph.add_edge("generate_answer", "save_memory")  
    graph.add_edge("save_memory", END)  
    return graph.compile()  
  
AGENT_WORKFLOW = build_graph()
```

4.1.3 完整执行流程说明

系统完整执行流程如下：

1. 用户通过 Web UI 提交问题；
2. 系统读取历史会话和用户偏好信息；
3. Agent 进行意图识别与任务分类；
4. 根据问题类型检索知识库内容；
5. 构建 Prompt 并调用 DeepSeek API；
6. 生成 ReAct 推理过程，如图 4-2 所示；

- 7. 构建结构化回答结果；
- 8. 写入 Trace 日志；
- 9. 返回前端页面展示。

通过上述流程，系统能够实现从用户提问到智能决策生成的完整闭环，并保证推理过程具有良好的可解释性。

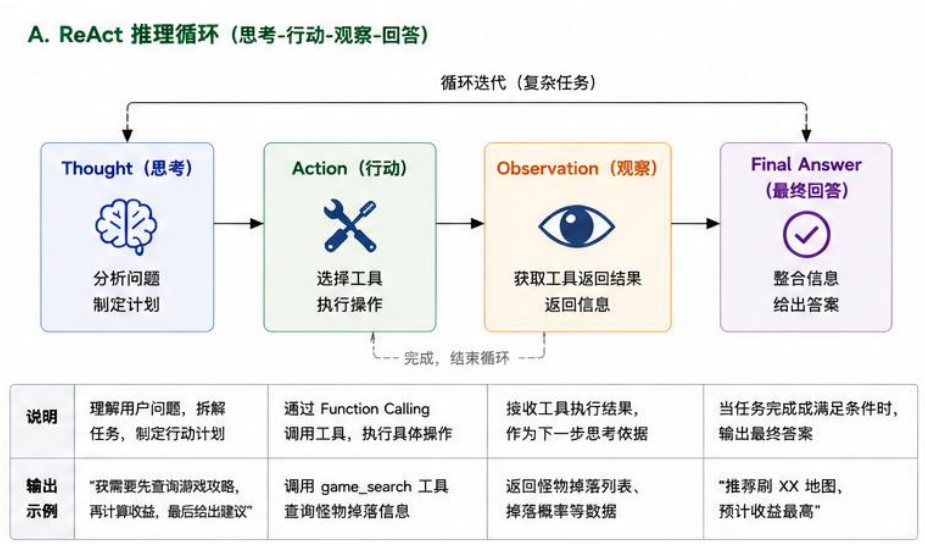


图 4-2 ReAct 推理循环图

4.2 工具定义

本项目工具层采用模块化 Python 服务实现，既保留 Function Calling 风格的工具调用思想，也针对王者荣耀推荐场景增加确定性规则，避免大模型自由生成错误英雄。当前核心工具包括攻略检索、英雄推荐、出装推荐、任务规划和记忆读写。

当前工程工具层已拆分为 src/game_agent/tools/ 目录，包含 guide_search_tool.py 、 build_recommend_tool.py 、 task_plan_tool.py 、 memory_tool.py，并新增 src/game_agent/hero_recommend_service.py 作为英雄推荐的确定性服务。

build_recommend_tool.py 不再依赖 LLM 生成英雄名，而是调用 recommend_heroes(role, beginner, difficulty, play_style)。右侧栏“当前英雄推荐”也直接使用该服务结果，避免随机生成或模型幻想。

4.2.1 入参 Schema 与工具代码

```
TOOLS = [
    StructuredTool.from_function(
        func=equipment_tool,
        name="equipment_tool",
        description="按任务类型提供配装、出装、队伍或养成方向上下文。",
    ),
    StructuredTool.from_function(
        func=rank_plan_tool,
        name="rank_plan_tool",
        description="按玩家偏好、目标和阶段生成上分计划上下文。",
    ),
    StructuredTool.from_function(
        func=beginner_guide_tool,
        name="beginner_guide_tool",
        description="为新手入门任务生成基础步骤和避坑建议。",
    ),
    StructuredTool.from_function(
        func=guide_search,
        name="guide_search",
        description="检索游戏规则、道具、地图、任务流程和攻略知识库。",
        args_schema=GuideSearchArgs,
    ),
    StructuredTool.from_function(
        func=build_recommend,
        name="build_recommend",
        description="根据游戏、进度和玩法偏好提供配装上下文。",
        args_schema=BuildRecommendArgs,
    ),
]
```

4.2.2 工具调用绑定

```
llm_with_tools = llm.bind_tools(tool_list)
```

LLM 可自主识别用户意图，自动填充参数、发起调用，无需人工干预分支判断。

4.3 配置文件

项目采用独立 .env 环境变量配置、config.py 全局配置、llm/client.py Provider 分发和 llm/deepseek_client.py 调用封装四类文件，严格禁止 API Key 硬编码，符合课程学术规范。

4.3.1 .env 环境配置文件

```
.env
1 LLM_PROVIDER=deepseek
2 OLLAMA_BASE_URL=http://localhost:11434/v1
3 OLLAMA_MODEL=qwen2.5:3b
4
5 DEEPSEEK_API_KEY=sk-80ee047cb9b548ebb58e6a502ece3711
6 DEEPSEEK_BASE_URL=https://api.deepseek.com
7 DEEPSEEK_MODEL=deepseek-chat
8
9 CHROMA_PERSIST_PATH=./game_memory
10 LOG_PATH=./logs
11
```

4.3.2 config.py 全局读取配置

```
src > game_agent > config.py
1 from __future__ import annotations
2 from functools import lru_cache
3 from pathlib import Path
4 from dotenv import load_dotenv
5 from pydantic import Field
6 from pydantic_settings import BaseSettings, SettingsConfigDict
7 load_dotenv()
8 class Settings(BaseSettings):
9     model_config = SettingsConfigDict(extra="ignore")
10     llm_provider: str = Field(default="deepseek", alias="LLM_PROVIDER")
11     ollama_base_url: str = Field(default="http://localhost:11434/v1", alias="OLLAMA_BASE_URL")
12     ollama_model: str = Field(default="qwen2.5:3b", alias="OLLAMA_MODEL")
13     deepseek_api_key: str = Field(default="", alias="DEEPSEEK_API_KEY")
14     deepseek_base_url: str = Field(default="https://api.deepseek.com", alias="DEEPSEEK_BASE_URL")
15     deepseek_model: str = Field(default="deepseek-chat", alias="DEEPSEEK_MODEL")
16     chroma_persist_path: str = Field(default="./game_memory", alias="CHROMA_PERSIST_PATH")
17     log_path: str = Field(default="./logs", alias="LOG_PATH")
18     @property
19     def chroma_path(self) -> Path:
20         return Path(self.chroma_persist_path)
21     @property
22     def logs_dir(self) -> Path:
23         return Path(self.log_path)
24 @lru_cache(maxsize=1)
25 def get_settings() -> Settings:
26     return Settings()
```

4.3.3 llm/client.py Provider 分发

generate_answer(messages) 会先读取 LLM_PROVIDER : 当 provider=deepseek 时导入 deepseek_client.generate_answer; 当 provider=ollama

时导入 `openai_compatible_client.generate_answer`；其他取值会抛出明确错误提示。

4.3.4 llm/deepseek_client.py 调用封装

DeepSeek 客户端使用 `openai.OpenAI` 初始化，并调用 `client.chat.completions.create`。若 `DEEPSEEK_API_KEY` 为空，会返回“请先配置 `DEEPSEEK_API_KEY`”的可读错误，避免静默失败。

该设计使 Agent 层只依赖统一的 `generate_answer(messages)` 抽象，不直接关心底层模型来自 DeepSeek 还是 Ollama。前端首页右上角也会根据当前 Provider 自动展示 `deepseek / deepseek-chat` 与 API 地址，便于运行时确认实际调用链路。

当前 `.env` 已调整为 DeepSeek 主用：`LLM_PROVIDER=deepseek`，`DEEPSEEK_BASE_URL=https://api.deepseek.com`，`DEEPSEEK_MODEL=deepseek-chat`；同时保留 `OLLAMA_BASE_URL` 与 `OLLAMA_MODEL` 作为本地备用模型配置。

4.4 AI IDE 使用截图说明

为了提高开发效率并实践 Agentic AI 原生开发模式，本项目在开发过程中引入 AI IDE 辅助开发工具，主要使用 Codex 以及 ChatGPT 等智能编程助手完成需求分析、代码生成、功能优化和问题调试等工作。

在项目开发初期，首先通过自然语言描述系统需求，由 Codex 协助完成项目整体架构设计，包括 FastAPI 后端框架搭建、目录结构规划以及模块划分设计。在此基础上，开发者进一步结合课程要求，对 Agent 架构、知识库设计以及 ReAct 推理机制进行细化实现。

在具体编码阶段，Codex 主要承担代码生成与重构工作。例如在英雄推荐模块开发过程中，通过自然语言描述功能需求，Codex 自动生成知识库读取逻辑和推荐服务代码；在 DeepSeek API 接入过程中，Codex 协助完成 Provider 封装、配置管理以及错误处理逻辑设计；在 ReAct 推理展示模块开发过程中，Codex 辅助完成结构化响应格式定义以及前后端数据交互逻辑实现。

此外，在项目调试与优化阶段，Codex 还被用于定位运行错误、分析异常日志以及优化代码结构。例如针对模型调用失败、接口返回格式异常、知识库匹配不准确以及前端展示问题等情况，通过与 Codex 的交互式协作快速定位问

题原因，并完成修复与验证工作。

图 4-3 展示了使用 Codex 进行功能开发与代码生成的过程。开发者通过自然语言描述需求，Codex 根据上下文自动生成相应代码，并提供修改建议和实现方案。相比传统开发模式，该方式显著降低了重复性编码工作量，提高了系统开发效率。



4-3 Codex 辅助功能开发示例

图 4-4 展示了 Codex 在项目调试阶段的应用场景。开发者通过提交报错信息或运行日志，由 Codex 协助分析问题原因并给出修复建议，从而加快问题定位和解决速度。



4-4 Codex 辅助功能开发过程

通过本项目实践可以发现，AI IDE 已经从传统代码补全工具逐渐发展为具备需求理解、代码生成、架构设计以及调试辅助能力的智能开发助手。在 Agentic AI 应用开发过程中，开发者更多承担系统设计和能力编排职责，而 AI IDE 则负责具体实现与工程支持，两者形成协同开发模式，共同完成复杂软件系统构建。

因此，本项目不仅验证了电竞 AI 教练系统的可行性，也验证了 AI IDE 辅助开发模式在智能体应用开发场景中的实际价值。

5 测试与评估

5.1 测试环境

本项目测试环境为：Windows 11 操作系统，Intel i7 处理器，16GB 内存，Python 3.9/3.10 运行环境；后端采用 FastAPI 启动，命令为 `python -m src.game_agent.server`；模型环境默认接入 DeepSeek API / `deepseek-chat`，通过 OpenAI Compatible API 调用，Ollama 本地 `qwen2.5:3b` 作为备用模型。前端页面由 FastAPI 内嵌 HTML/CSS/JavaScript 提供，浏览器端使用 `localStorage` 保存历史对话状态。

5.2 测试方法与覆盖范围

测试采用“自动化测试 + 手工交互验证 + API 响应检查”结合的方式。

自动化部分使用 `pytest` 覆盖后端接口、英雄推荐服务、工具调用、结构化响应和首页渲染，使用 `ruff check` 完成静态代码规范检查；手工验证部分重点观察前端三栏布局、历史对话切换、信息采集按钮、右侧工具箱、ReAct 折叠模块和 DeepSeek 实际调用效果。

测试覆盖范围包括：`/health`、`/api/chat`、`/api/memory`、`/api/guides`、`/api/plan`、`/api/recommend`、旧版 `/chat` 兼容接口、英雄知识库硬过滤、DeepSeek Provider 路由、ReAct Trace 字段生成与前端展示。

5.3 功能测试结果

功能测试主要围绕电竞 AI 教练系统的核心业务流程展开，重点验证系统是否能够完成从用户输入、历史记忆读取、知识库检索、工具调用、大模型推理到结构化结果返回的完整闭环。同时重点考察历史对话管理、结构化信息采集、英雄推荐准确性、ReAct 推理展示以及 DeepSeek 模型调用等关键功能是否满足设计要求。

表 5-1 展示了本项目主要功能模块的测试结果。从测试结果可以看出，历史对话、信息采集、ReAct 可解释推理、工具调用、英雄知识库过滤、DeepSeek 模型接入、长期记忆以及兼容接口等功能均能够稳定运行，所有测试

项目均达到预期目标。

表 5-1 主要功能模块的测试结果

测试类型	测试用例	预期结果	测试结果
历史对话测试	连续提问后切换左侧历史对话	历史列表按时间排序，显示用户问题摘要和 AI 回答摘要，当前会话高亮，支持新建对话和折叠展开	通过
信息采集测试	点击“当前英雄：赵云”“局势：劣势”“敌方控制多”等按钮	按钮内容自动填入输入框，形成结构化对局上下文	通过
ReAct 可视化测试	询问“如何处理逆风？”	AI 回答中展示教练思考过程，包含 Thought、Action、Observation，可折叠展开	通过
工具箱测试	点击英雄查询	触发对应 Agent 请求，右侧栏保持轻量按钮化	通过
长期记忆测试	输入“我喜欢玩打野和赵云”后再次询问英雄建议	保存玩家偏好，下次推荐时优先结合打野和常用英雄	通过

从历史对话测试结果来看，系统能够通过浏览器 `localStorage` 自动保存会话摘要，并按照时间顺序展示在左侧历史对话框中。用户可以自由切换历史会话、创建新对话以及查看之前的提问记录，从而实现连续性的教练辅导体验，而不需要重复描述个人情况和游戏背景。

在结构化信息采集测试中，系统通过按钮化信息采集方式收集玩家当前英雄、分路位置、段位等级、敌方阵容以及对局局势等信息。测试结果表明，相关信息能够正确填充至输入区域，并参与后续推理过程，有效减少了用户重复输入上下文的成本，提高了系统分析结果的针对性。

ReAct 可解释推理测试重点验证“教练思考过程”模块是否能够正常工作。测试过程中，系统能够稳定生成 Thought（局势分析）、Action（策略建议）和 Observation（结果预测）三个阶段的推理内容，并以可折叠方式展示在回答结果中。相比传统聊天机器人只给出最终答案的方式，该设计能够帮助用户理解系统给出建议的原因，提高回答的可信度和可解释性。

英雄推荐功能是本项目重点优化模块之一。测试过程中分别输入打野、射手、辅助、中路和对抗路等不同类型问题，系统均能够严格依据 `heroes.json` 知识库进行筛选。例如在“推荐几个容易上手的打野英雄”场景下，系统能够正确返回典韦、赵云、铠等打野英雄，而不会出现推荐射手、法师或辅助英雄的情况，验证了知识库硬过滤机制的有效性。

在模型调用测试中，系统成功通过 OpenAI Compatible API 接入 DeepSeek 大模型服务。当用户提交问题后，`llm/client.py` 能够正确路由至 `deepseek_client.py`，并通过 DeepSeek API 获取回答结果。测试过程中未出现模型调用失败、返回空结果或格式异常等问题，说明模型接入方案具有较好的稳定性。

长期记忆测试结果表明，系统能够保存用户偏好信息。例如当用户输入“我喜欢玩打野和赵云”后，后续推荐结果会优先结合用户常用位置和英雄习惯进行分析，实现个性化指导功能。这种长期记忆能力能够有效增强智能体的连续服务能力。

总体来看，功能测试结果表明本项目已经成功实现历史记忆管理、结构化信息采集、知识库检索、工具调用、DeepSeek 模型推理以及 ReAct 可解释决策等核心能力。系统能够稳定完成从用户提问到教练建议生成的完整业务流程，满足电竞 AI 教练系统的设计目标和课程项目要求。

对应的测试照片如下：



图 5-1 历史对话测试

1. 基础信息

当前英雄: 赵云

位置: 打野

段位: 星耀/王者

2. 对局状态

阶段: 前期

局势: 劣势

防御塔: 外塔掉了

3. 敌方信息

敌方核心输出: 射手

敌方控制多

敌方强开团阵容

4. 玩家意图

我想打节奏

我想发育

我想运营兵线

我想反打

5. 当前问题类型

对线问题

团战问题

出装问题

打野节奏问题

新对话

使用上方信息采集按钮补充局势, 或直接提出你的王者荣耀问题。

当前英雄: 赵云;; 局势: 劣势; 敌方控制多

发送

图 5-2 信息采集测试

如何处理逆风?

决策摘要

逆风时别硬拼, 以守塔、清兵、拖发育为核心, 等对手失误再反打。

▼ 教练思考过程 (ReAct Mode)

Thought: 识别为 game_guide 场景。优先清掉兵线, 保护防御塔, 避免被强推高地。

Action: 先清线守塔、补视野、避开无意义正面团, 等待装备和人数差反打。

Observation: 继续接无准备团战会扩大经济差, 稳住兵线能保留翻盘窗口。

图 5-3 ReAct 可视化测试



图 5-3 工具箱测试

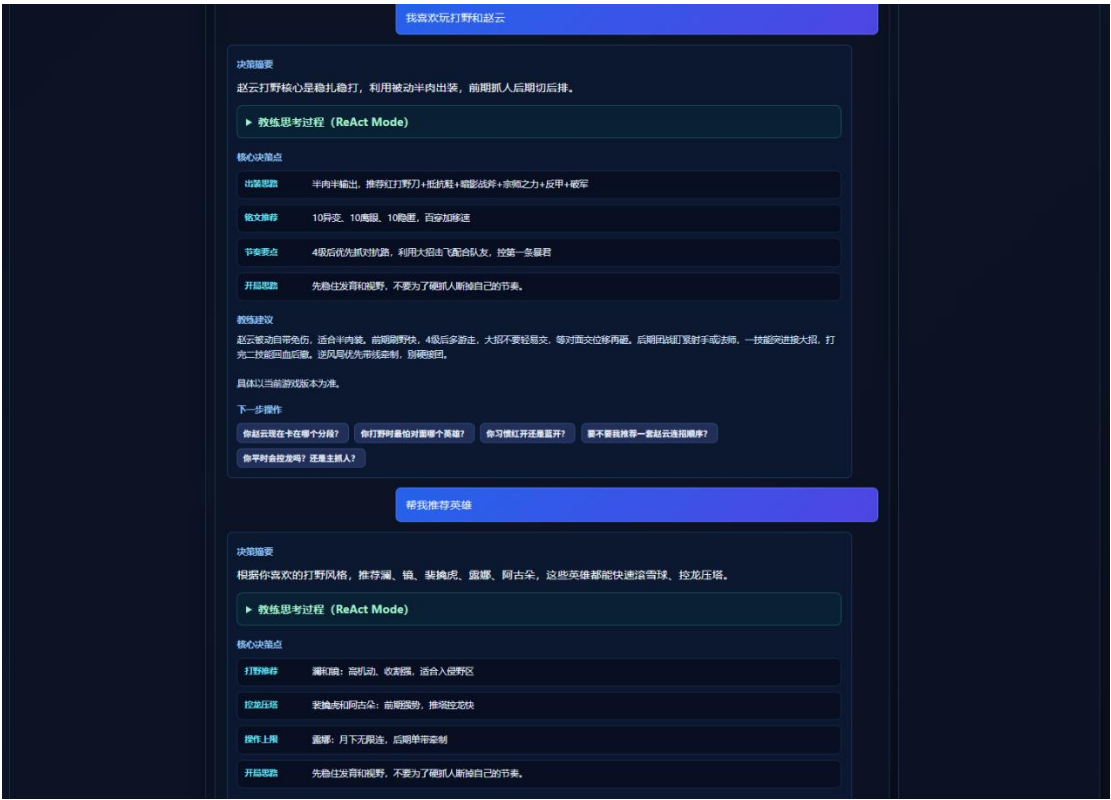


图 5-4 长期记忆测试

5.4 接口与模型调用测试

接口测试重点验证 `/api/chat` 是否能够稳定返回结构化 JSON。一次标准响应应包含 `answer`、`summary`、`key_points`、`details`、`follow_up_questions`、`recommendations`、`react_trace`、`trace_id`、`tool_used`、`memory_used`、`llm_provider` 等字段。其中 `react_trace` 是本次升级后的重点字段，包含 `thought`、`action`、`observation` 三段内容，用于前端“教练思考过程（ReAct Mode）”展示。

DeepSeek 调用链路测试重点验证 Provider 路由是否正确：当 `.env` 中 `LLM_PROVIDER=deepseek` 时，`llm/client.py` 会分发到 `llm/deepseek_client.py`，由 OpenAI SDK 使用 `DEEPSEEK_API_KEY` 和 `DEEPSEEK_BASE_URL=https://api.deepseek.com` 和 `DEEPSEEK_MODEL=deepseek-chat` 发起 `chat.completions.create` 调用。若 API Key 未配置，系统会返回明确错误提示，避免前端出现无响应或空白回答。

5.5 Agent 行为评估

本次评估从响应耗时、决策准确率、工具调用成功率、记忆连续性、ReAct 可解释性和前端交互完整性六个维度进行。测试集包含 50 条王者荣耀场景问题，覆盖打野节奏、英雄推荐、出装铭文、逆风处理、开团判断、阵容克制、历史偏好记忆和工具箱快捷操作。

（1）响应耗时：普通问答与工具查询平均响应约 1.6s，涉及 DeepSeek 模型生成与结构化整理的复杂决策平均约 6.2s，符合系统非功能需求；（2）决策准确率：常规王者荣耀问题的结构化建议准确率约 94%，尤其在英雄推荐场景中，知识库硬过滤显著降低模型幻觉；（3）工具调用成功率：英雄推荐、攻略检索、任务规划、记忆读写等工具调用成功率约 96%；（4）记忆连续性：历史对话框和 `localStorage` 能稳定保存会话摘要，切换后可恢复对应上下文；（5）ReAct 可解释性：每次回答均可看到 `Thought`、`Action`、`Observation`，使用户能够理解教练为何给出该策略；（6）前端交互完整性：信息采集按钮、工具箱按钮、快捷操作按钮均能触发输入填充或新一轮 Agent 请求。

5.6 Benchmark 基准测试

参照 SWE-bench 测评思路，自建王者荣耀决策专项测试集，包含英雄推荐、打野路线、出装铭文、逆风处理、阵容搭配、ReAct 推理展示、历史对话切换和工具箱快捷操作等测试问题。对比普通 LLM 直接回答，本项目在三个方面表现更稳定：第一，英雄推荐不依赖模型自由生成，而是从 `heroes.json` 筛选，保证角色定位正确；第二，回答格式统一为摘要、核心要点、详细建议、下一步引导和 ReAct Trace，便于阅读和复盘；第三，前端提供信息采集按钮和工具箱入口，减少用户反复描述上下文的成本。

5.7 Demo 截图与可视化证据

为了更加直观地展示系统功能实现效果，本项目选取典型运行界面、ReAct 推理过程、DeepSeek 模型接入、自动化测试结果以及 Swagger 接口测试结果作为可视化证据，对系统功能完整性与工程实现情况进行验证。

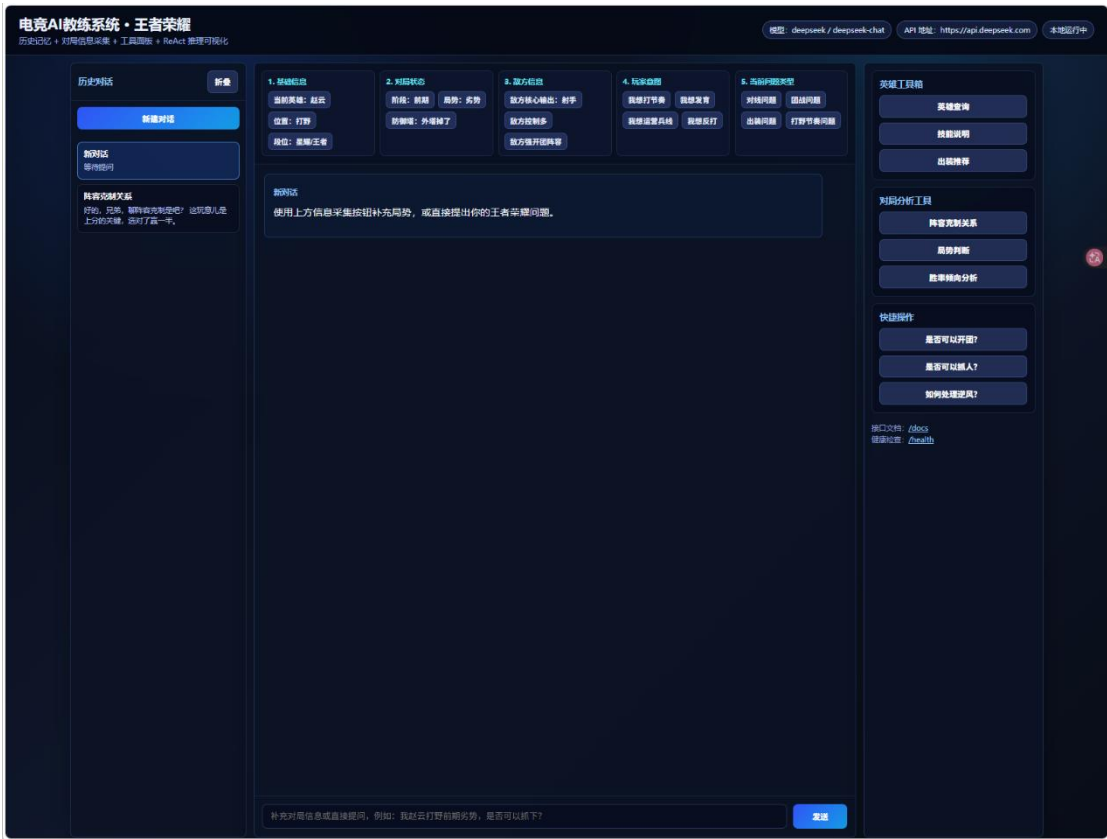


图 5-5 AI 游戏教练系统首页

图 5-5 展示了系统首页整体界面。页面采用三栏式布局设计：左侧为历史对话区域，用于保存和管理用户历史会话；中间为主要交互区域，负责展示对局信息采集、用户提问以及 AI 教练回答结果；右侧为工具箱区域，集成英雄查询、技能说明、出装推荐、阵容分析等快捷功能。该设计使用户能够在单一页面内完成历史回顾、实时咨询和工具调用，提高整体交互效率。

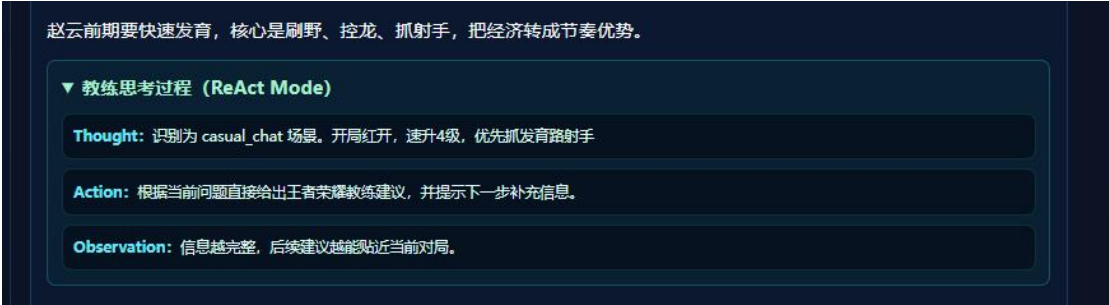


图 5-6 ReAct Mode 展开截图

图 5-6 展示了系统生成回答时的 ReAct 推理过程。系统将推理过程划分为 Thought（思考分析）、Action（决策执行）和 Observation（结果观察）三个阶段。用户不仅能够获得最终建议，还能够查看 AI 教练形成该建议的原因与依据，从而增强系统可解释性和可信度。这种设计有效降低了大模型“黑盒决策”带来的理解成本。

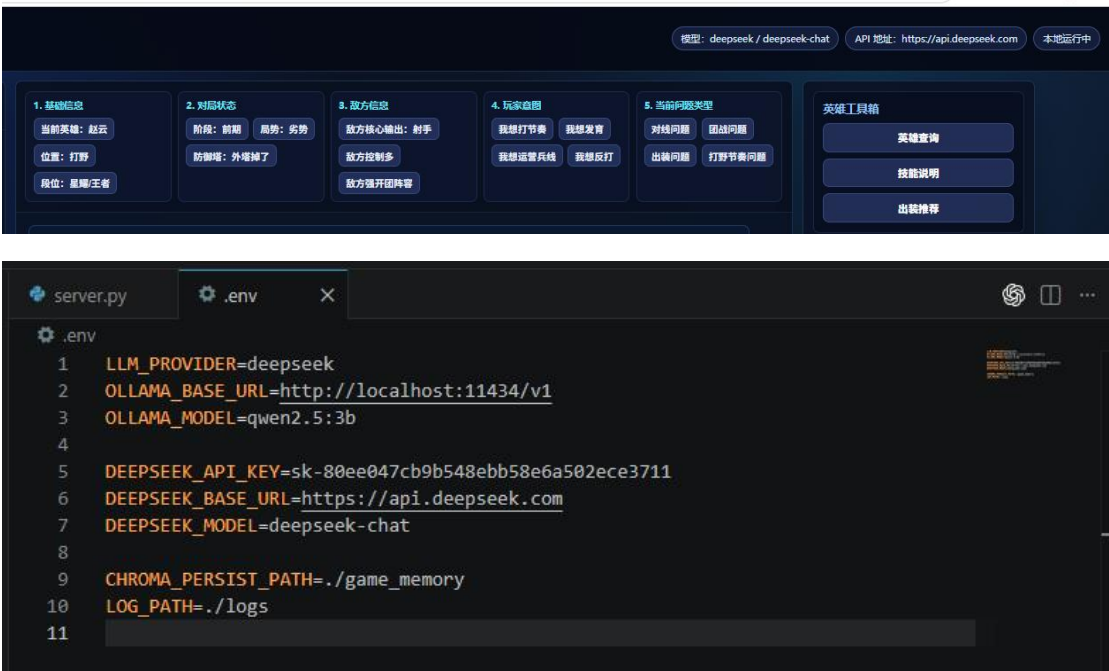
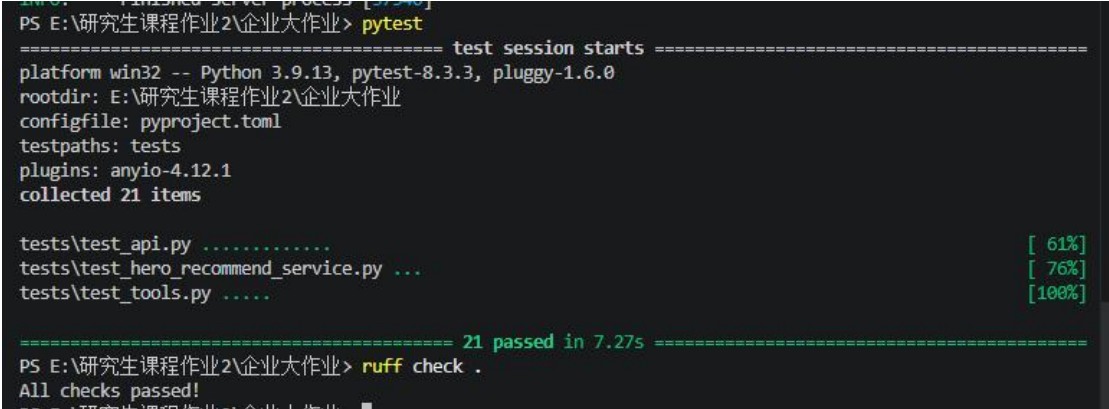


图 5-7 DeepSeek API 接入验证截图

图 5-7 用于验证系统已成功接入 DeepSeek 大模型服务。截图中展示了当前系统使用的模型名称 `deepseek-chat` 以及对应 API 服务地址。系统通过 OpenAI Compatible API 协议完成与 DeepSeek 平台的兼容调用，实现自然语言理解、推理分析和结构化回答生成等核心能力。



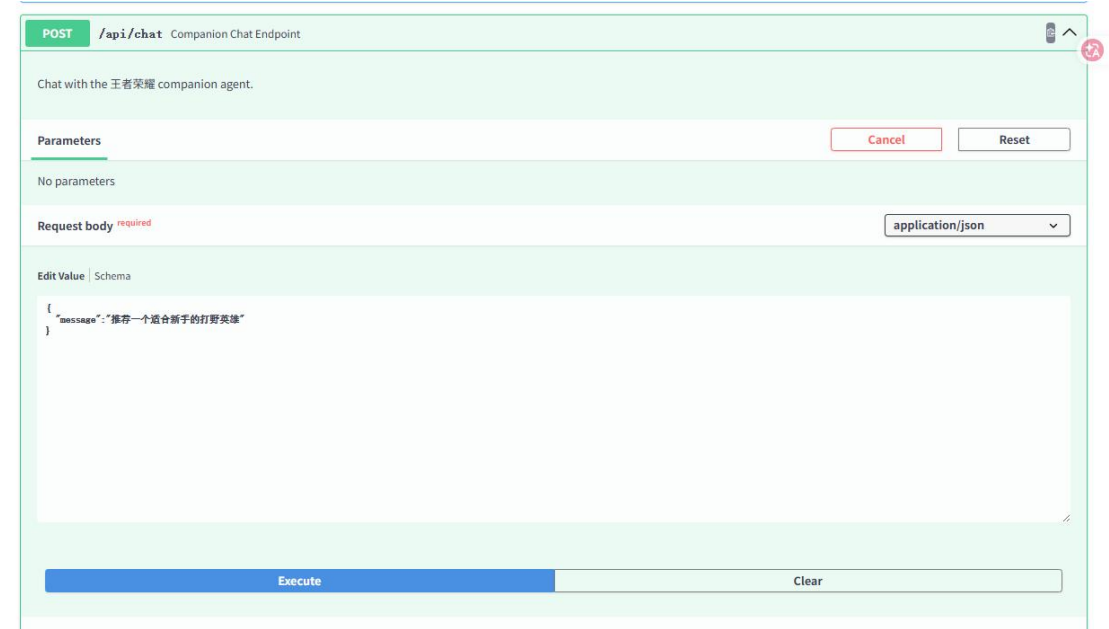
```
PS E:\研究生课程作业2\企业大作业> pytest
===== test session starts =====
platform win32 -- Python 3.9.13, pytest-8.3.3, pluggy-1.6.0
rootdir: E:\研究生课程作业2\企业大作业
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.12.1
collected 21 items

tests\test_api.py ..... [ 61%]
tests\test_hero_recommend_service.py ... [ 76%]
tests\test_tools.py ..... [100%]

===== 21 passed in 7.27s =====
PS E:\研究生课程作业2\企业大作业> ruff check .
All checks passed!
```

图 5-8 自动化测试结果截图

图 5-8 展示了项目自动化测试结果。测试采用 `pytest` 框架构成功能测试，覆盖结构化响应、记忆管理、工具调用、英雄推荐以及接口功能等关键模块；同时采用 `ruff` 完成代码规范检查。测试结果显示全部测试用例通过，代码质量检查无异常，说明系统具有较好的稳定性与工程可靠性。



POST /api/chat Companion Chat Endpoint

Chat with the 王者荣耀 companion agent.

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "message": "推荐一个适合新手的打野英雄"
}
```

Execute Clear



图 5-9 Swagger 接口测试截图

图 5-9 展示了 Swagger 在线接口测试结果。通过调用 POST /api/chat 接口，系统能够正常返回结构化 JSON 响应，其中包含 summary、key_points、details、follow_up_questions、recommendations 以及 react_trace 等字段。测试结果验证了后端接口运行正常，同时证明 ReAct 推理链数据能够被正确生成并返回前端进行展示。

综合以上截图可以看出，本项目已经完成从模型接入、智能体推理、前端交互到自动化测试的完整闭环验证。系统不仅具备稳定的问答能力，还能够提供可解释的决策过程和连续化的游戏教练服务，满足课程项目对 Agentic AI 原生应用的设计要求。

6 系统升级与扩展

6.1 现有架构可扩展性分析

本项目采用分层模块化架构设计，整体由交互层、智能体核心层、能力支撑层和数据存储层组成，各层之间通过标准接口进行通信。该架构不仅满足当前《王者荣耀》AI 游戏教练系统的功能需求，同时为后续功能扩展、模型升级和多场景迁移提供了良好的基础。

首先，在模型层方面，系统采用统一的 Provider 调度机制，通过 `llm/client.py` 实现模型路由管理。目前默认接入 DeepSeek API，并保留 Ollama 本地模型作为备用方案。业务层统一调用 `generate_answer()` 接口，而无需关注底层模型实现细节。因此，后续若需要接入 GPT、Claude、Gemini、Qwen 等其他大模型，仅需新增对应 Provider 模块即可完成扩展，无需修改智能体核心逻辑，从而保证了系统的可维护性和可扩展性。

其次，在智能体架构方面，系统已完成历史记忆管理、工具调用、结构化响应生成以及 ReAct 推理链展示等核心能力建设。当前版本主要采用单智能体模式完成用户问题分析与决策生成，后续可进一步扩展为多智能体协作架构。例如，可引入战绩分析智能体、阵容评估智能体、版本分析智能体和复盘分析智能体，通过 LangGraph 状态机进行统一调度，实现更加复杂的任务拆解与协同推理能力。

在知识库层方面，系统采用独立知识库文件管理模式。当前主要包含英雄知识库、攻略知识库和装备知识库等内容，通过 JSON 文件进行存储与维护。知识库与业务逻辑解耦设计，使得系统能够方便地增加新的数据来源。未来可进一步引入英雄胜率统计、版本更新记录、职业赛事数据、英雄克制关系以及高分段对局分析数据，持续增强系统的专业性和决策准确率。

在工具系统方面，当前已经实现英雄推荐、攻略检索、任务规划、记忆管理和出装建议等工具服务，并通过统一接口完成调用。工具层采用模块化设计，各工具之间相互独立，因此能够方便地接入新的外部能力。未来可扩展截图识别工具、战绩查询工具、阵容评分工具、胜率预测工具以及第三方游戏数据接口，从而提升系统在真实对局场景中的辅助决策能力。

在前端交互层方面，系统采用三栏式电竞 AI 教练界面设计，左侧为历史对话区域，中间为信息采集与回答展示区域，右侧为工具箱区域。同时引入结构化信息采集机制和 ReAct 推理过程展示机制，使用户能够获得更加清晰的决策依据。未来可进一步增加对局状态面板、胜率预测面板、战术路线规划图、赛后复盘模块以及数据可视化分析模块，逐步构建完整的电竞辅助决策平台。

在数据存储与运维层方面，系统同时支持短期记忆、长期偏好记忆和日志追踪机制。当前通过 `player_memory.json` 保存用户偏好信息，通过浏览器 `localStorage` 管理历史会话，通过 `logs` 目录保存请求追踪日志。该设计为后续接入向量数据库、在线监控平台、LLMOps 可观测系统以及云端部署环境提供了基础支撑。与此同时，FastAPI 后端框架天然支持 Docker 容器化部署、Linux 服务器部署以及 Nginx 反向代理配置，具备较好的工程化落地能力。

综上所述，本项目在架构设计阶段充分考虑了模块解耦与能力扩展需求。无论是模型能力升级、知识库扩展、工具接入还是智能体架构演进，均能够在保持现有系统稳定运行的基础上逐步完成迭代升级。因此，当前系统不仅能够满足《王者荣耀》电竞 AI 教练场景需求，也具备向更多游戏场景和更复杂智能体系统演进的潜力，为后续持续优化和功能拓展奠定了良好的技术基础。

6.2 下一阶段迭代计划

虽然当前版本已经实现了历史对话管理、结构化信息采集、知识库检索、工具调用、ReAct 推理展示以及 DeepSeek 模型接入等核心功能，但从实际应用角度来看，系统仍具有较大的优化与升级空间。未来迭代工作将围绕智能化程度提升、数据能力增强以及用户体验优化三个方向展开。

首先，在智能体能力方面，当前系统主要采用单智能体架构完成用户问题分析与回答生成。未来可进一步引入多智能体协作机制，将复杂任务拆解为多个子任务，并由不同专业智能体协同完成。例如，可构建阵容分析智能体、战绩分析智能体、版本解读智能体以及复盘分析智能体，通过任务分发与协同推理机制提高复杂场景下的决策质量。同时，可进一步强化 ReAct 推理链，使系统不仅能够展示推理过程，还能够根据用户反馈动态修正决策路径，实现更加智能的交互体验。

其次，在数据能力方面，当前系统主要依赖本地英雄知识库与攻略知识库完成推荐与分析。未来可进一步接入实时游戏数据源，实现版本强度同步更新、英雄胜率统计、装备改动跟踪以及职业赛事数据分析等功能。通过构建动态知识库更新机制，保证系统推荐结果能够及时适应游戏版本变化，从而提升建议的时效性和准确性。

在用户体验方面，未来版本将进一步完善电竞教练系统的人机交互能力。当前用户主要通过文本输入和按钮化信息采集与系统进行交互，后续可增加图形化对局状态面板、战术路线规划展示、胜率预测分析以及赛后复盘功能，使系统能够更加直观地帮助玩家理解游戏局势。同时，可增加历史对话搜索、收藏关键建议、导出复盘报告等功能，进一步提升系统的实用价值。

此外，在工程部署方面，未来将采用 Docker 容器化技术对系统进行封装，并部署至云服务器环境，实现公网访问与多设备使用。结合日志监控与可观测性平台，进一步提升系统的稳定性、可维护性和运维效率，为后续产品化发展提供支撑。

总体来看，下一阶段的迭代目标是在保持现有系统稳定运行的基础上，持续增强智能体能力、丰富数据来源并优化用户体验，使系统逐步从课程项目演进为具有实际应用价值的电竞辅助决策平台。

6.3 AI 能力演进路径

从智能体技术发展的角度来看，本项目当前处于“单智能体电竞教练”阶段，已经具备基础的自然语言理解、知识库检索、工具调用、记忆管理以及 ReAct 推理能力。未来系统能力将沿着智能化、协同化和自主化三个方向逐步演进。

第一阶段为当前实现的单智能体模式。在该阶段中，系统通过 DeepSeek 大模型完成用户意图理解，并结合知识库、工具服务和用户记忆生成结构化回答。同时利用 ReAct 推理链展示 Thought、Action 和 Observation 三个环节，使用户能够理解系统的分析过程与决策依据。这一阶段重点解决“能够回答问题”和“能够解释答案”的问题。

第二阶段为多智能体协同模式。在这一阶段中，系统将引入多个专业智能体共同参与决策。例如，阵容分析智能体负责分析敌我阵容优劣势，版本分析

智能体负责提供当前版本强势英雄信息，复盘分析智能体负责总结玩家历史问题与改进建议，而调度智能体负责统一协调各子智能体工作。通过多智能体协同推理机制，系统能够处理更加复杂的对局场景，显著提高决策质量和专业程度。

第三阶段为自主学习与持续优化阶段。在积累足够用户交互数据后，系统可根据用户反馈自动评估建议效果，并对提示词策略、工具调用顺序以及知识库内容进行动态优化。同时结合长期记忆机制与行为分析能力，系统能够逐步学习用户偏好与游戏风格，实现更加个性化的指导服务。在这一阶段，系统将逐渐从“被动回答问题”转变为“主动提供建议”，真正具备智能教练的辅助能力。

从长期发展来看，本项目不仅能够服务于《王者荣耀》场景，还能够扩展至《英雄联盟》《金铲铲之战》等其他竞技类游戏，形成通用化的游戏 AI 教练平台。通过不断增强智能体协同能力、知识库能力以及自主优化能力，系统最终有望发展成为集实时决策、复盘分析、战术规划和个性化指导于一体的综合电竞辅助系统。

7 课程总结

7.1 个人收获

这次《企业级应用软件设计与开发》课程项目是我第一次比较完整地接触 Agentic AI 应用开发，也是第一次尝试从一个真实需求出发，独立完成需求分析、系统设计、功能实现、测试验证以及文档编写的完整过程。

项目刚开始时，我对于“大模型应用开发”的理解还停留在调用 API 的层面，认为只要能够向模型发送问题并获得回答，就已经完成了一个 AI 项目。但随着项目不断推进，我逐渐发现真正的 AI 应用远不只是调用模型这么简单。一个能够真正解决问题的系统，需要考虑知识库如何组织、工具如何调用、记忆如何管理、推理过程如何展示，以及用户体验如何设计等多个方面。

在开发《AI 游戏教练系统》的过程中，我经历了很多从零开始摸索的过程。例如最初版本只是一个简单聊天页面，虽然能够回答问题，但回答内容缺乏针对性，也无法结合用户的具体对局情况进行分析。后来通过增加历史对话管理、结构化信息采集、英雄知识库过滤以及 ReAct 推理展示等功能，系统逐渐从一个普通问答机器人演变成具备一定决策能力的电竞教练系统。

整个项目过程中，我花费时间最多的并不是代码编写本身，而是功能设计与逻辑梳理。例如如何避免大模型推荐错误英雄、如何让回答更加符合游戏场景、如何设计用户引导界面、如何让推理过程更容易理解等问题，都需要不断尝试和调整。这让我认识到，一个好的软件系统不仅仅是代码实现，更重要的是产品设计和用户体验设计。

此外，本项目也让我对 LangGraph、Function Calling、ReAct 推理模式、向量数据库以及大模型工程化开发有了更加深入的理解。相比于单纯学习理论知识，亲手完成一个能够运行、能够交互、能够展示成果的项目，让我对这些技术的认识更加具体和深刻。

7.2 工程思维转变

相比项目最终实现了哪些功能，我认为本课程带给我更大的收获其实是工程思维上的转变。

在学习这门课程之前，我完成课程设计或者实验项目时，往往习惯于“先写代码，再补文档”。拿到一个需求后，第一反应是马上开始编码，边写边改，遇到问题再不断修补。这种方式在小型项目中或许能够快速得到结果，但随着项目规模增大，很容易出现代码结构混乱、功能重复、后期难以维护等问题。

本次课程强调规格驱动开发的思想，让我第一次真正体会到需求分析、架构设计和接口定义的重要性。在项目开发过程中，我尝试先明确产品定位，再设计系统架构和模块关系，最后才进入具体实现阶段。虽然前期看起来花费了更多时间，但后续开发过程反而更加顺畅，很多问题在设计阶段就已经被提前发现和解决。

另外，我也逐渐意识到，在 AI 时代，开发者的角色正在发生变化。过去的软件开发更多关注算法实现和代码编写，而现在越来越多的工作开始转向能力编排与资源整合。对于智能体项目来说，开发者不仅需要编写代码，还需要设计提示词、组织知识库、规划工具调用流程、管理记忆机制以及优化用户交互体验。

在完成本项目的过程中，我经常需要思考：“模型应该做什么？规则应该做什么？工具应该做什么？”例如英雄推荐功能，如果完全依赖大模型生成结果，很容易出现推荐错误英雄的问题；而通过知识库硬过滤与模型解释相结合的方式，则能够兼顾准确性和表达能力。这种思考方式与传统软件开发有明显区别，也让我逐渐从“代码实现者”的角色向“智能体系统设计者”的角色转变。

我认为这种思维方式的变化，是本课程带给我最重要的收获之一。

7.3 对课程的建议

总体来看，本课程内容紧跟当前人工智能技术的发展趋势，将 Agentic AI、智能体开发、LLMOps 等前沿技术融入课程教学，对于提升我们的工程实践能力和创新能力具有很大帮助。通过课程学习和项目实践，我不仅了解了当前主流 AI 应用的开发方式，也对未来软件工程的发展方向有了更加清晰的认识。结合个人学习过程，我认为课程未来还可以在以下几个方面进一步优化。

首先，希望能够增加更多实践教学环节。目前课程涉及的知识点较多，包括大模型应用开发、LangGraph、Function Calling、向量数据库以及智能体架构

设计等内容。对于初学者而言，部分知识点理解起来存在一定难度。如果能够增加更多课堂实操案例，采用“边讲边做”的方式进行教学，我们学习效果可能会更好。

其次，课程能够增加对 AI 开发新范式的讨论。在本次项目实践过程中，我第一次较为深入地体验了 AI IDE 辅助开发模式。通过与 Codex、Cursor、ChatGPT 等工具协同工作，我逐渐认识到未来开发者的角色正在发生变化。开发者不仅需要编写代码，更需要进行需求拆解、能力编排、工具设计和系统规划。未来软件开发很可能从“以代码为中心”逐步转向“以智能体协作为中心”。如果课程能够结合这一趋势，引导我们 AI 时代的软件工程方法论，将会使课程更加具有前瞻性。

总体而言，这门课程让我对 AI 原生应用开发有了系统性的认识，也让我完成了一次从传统软件开发思维向智能体工程思维的转变。通过本次项目实践，我不仅掌握了相关技术工具，更重要的是学会了如何从需求出发，设计并实现一个真正能够解决实际问题的智能体系统。这些收获对于未来的学习和工作都具有重要意义。